

e.g., York University
e.g., Lassonde School of Engineering
e.g., Earth & Space Science & Engineering

Assignment Title

Course Code: Course Name

Student:

Your Name

123456789

email@my.yorku.ca

Instructor:

Dr. Professor

Due date

LOGO

Abstract

This document is a beginner-friendly guide to this template. Each section introduces one or more packages included in the template, explains what it does in plain terms, and shows a simple working example. To get the most out of this file, open the compiled PDF and the source `demo.tex` file side by side. Reading the code while looking at the result it produces is the fastest way to understand how each feature works.

Contents

Abstract	ii
List of Figures	viii
List of Tables	viii
List of Algorithms	ix
List of Listings	ix
1 Conditionals & Utilities	1
1.1 xifthen	1
1.2 xstring	1
1.3 lipsum	1
2 Colours	2
2.1 xcolor	2
2.1.1 Setting the Theme Colour	2
2.1.2 Custom Defined Colours	2
2.1.3 Built-in Named Colours	3
2.1.4 Theme Colour and Tints	3
2.1.5 Using \textcolor and \colorbox	4
3 Page Geometry	5
3.1 geometry	5
3.2 setspace	5
3.3 pdflscape	6
4 Language & Font	7
4.1 babel	7
4.2 fontspec	7
4.2.1 \metaSerifFont and \seriftext	7
4.2.2 \metaSansFont and \sansseriftext	7
4.2.3 Default Font	8

5	Spacing Around Headings	8
5.1	titlesec	8
5.1.1	Heading Levels	8
5.1.2	Abstract environment	9
6	Math	9
6.1	amsmath, amssymb, amsfonts	9
6.1.1	equation	9
6.1.2	align	10
6.1.3	gather	10
6.1.4	multline	10
6.1.5	flalign	10
6.1.6	alignat	11
6.1.7	split	11
6.1.8	cases	11
6.2	mathtools	11
6.3	bm	12
6.4	siunitx	12
7	Chemistry	12
7.1	mhchem	12
7.2	Reaction Environments	12
7.2.1	requation	13
7.2.2	ralign	13
7.2.3	rgather	13
7.2.4	rmultline	14
7.2.5	rflalign	14
7.2.6	ralignat	14
7.2.7	split	14
8	Theorem Environments	15
8.1	amsthm	15
8.1.1	Plain — Theorem, Lemma, Proposition, Corollary, Conjecture, Axiom, Proof	15
8.1.2	Definition — Definition, Example, Criterion	15
8.1.3	Remark — Remark, Note	15

9	Figures	16
9.1	graphicx	16
9.2	float	16
9.3	subcaption	16
9.4	caption	16
9.5	pdfpages	17
10	Tables	17
10.1	booktabs	17
10.2	array	18
10.3	tabularx	18
10.4	multirow	19
10.5	ltablex	19
11	Diagrams & Plots	20
11.1	tikz	20
11.2	pgfplots	20
11.3	circuitikz	21
11.4	pgf-pie	22
11.5	environ	22
12	Algorithms	22
12.1	algorithm and algpseudocode	22
13	Links & References	24
13.1	hyperref	24
13.2	cleveref	24
14	Typography	26
14.1	microtype	26
14.2	xurl	26
14.3	indentfirst	26
14.4	ragged2e	27
15	Lists	27
15.1	enumitem	27

16 Problem & Solution Environments	28
16.1 tcolorbox	28
17 Code Listings	29
17.1 listings	29
17.2 minted	30
17.3 codelisting Environments	30
17.3.1 MATLAB	30
17.3.2 Java	31
17.3.3 Python	31
17.3.4 C++	31
17.3.5 LaTeX	31
18 Header & Footer	32
18.1 fancyhdr	32
18.1.1 Smart Collision Logic	32
19 Bibliography	33
19.1 csquotes	33
19.2 biblatex	33
References	36
A Appendices	37
A.1 Counter Resets	37
A.2 Section Heading Format	37
A.3 Cross-Referencing Appendices	37
B Metadata Reference	39
B.1 Submission Information	39
B.2 Title Page	40
B.3 Bibliography	40
B.4 Font & Spacing	41
B.5 Colours	41
B.6 Numbering	42
B.7 Drafting	42

C	Side-by-Side Layouts with <code>minipage</code>	44
C.1	Text Side by Side	44
C.2	Display Math Side by Side	44
C.3	Proof Step Comparison	44
C.4	Figure and Caption Side by Side	44
D	Notation	46
D.1	General Mathematics	46
D.2	Algebra	46
D.3	Physics	46
E	Greek Letters Reference	48
F	Common Math Operators and Symbols	49
F.1	Logic and Sets	49
F.2	Relations	49
F.3	Arrows	49
G	Font Commands	50
G.1	Text Font Shapes and Weights	50
G.2	Font Sizes	50

List of Figures

9.1	Faculty logo included with <code>\includegraphics</code> .	16
9.2	Two sub-figures using <code>subcaption</code> . (a) First sub-figure. (b) Second sub-figure.	16
11.1	Simple block diagram drawn with <code>tikz</code> .	20
11.2	Sine and cosine waves plotted with <code>pgfplots</code> .	21
11.3	Bar graph plotted with <code>pgfplots</code> .	21
11.4	Simple RC circuit drawn with <code>circuitikz</code> .	21
11.5	Annual budget allocation across departments.	22
C.1	First independent figure.	45
C.2	Second independent figure.	45

List of Tables

1.1	Common <code>xstring</code> operations.	1
2.1	Accepted values for <code>\metaThemeColour</code> .	2
2.2	Custom colours defined in <code>stemtemplate.sty</code> .	3
2.3	Built-in <code>xcolor</code> named colours.	3
2.4	Active theme colour, tints, shades, and blends.	4
3.1	Wide data table spanning landscape page.	6
5.1	Heading levels and their appearance in this template.	8
6.1	SI unit examples using <code>siunitx</code> .	12
9.1	Common <code>\includepdf</code> options.	17
10.1	Basic table using <code>booktabs</code> .	17
10.2	Fixed-width centred columns using <code>array</code> .	18
10.3	Common float placement specifiers.	19
10.4	Table with multi-row cells using <code>multirow</code> .	19
10.5	Table packages and their purposes, demonstrating <code>ltablex</code> with an L column.	19
12.1	Common <code>algpseudocode</code> commands.	23
13.1	Cross-reference examples using <code>\cref</code> for lowercase and <code>\Cref</code> for uppercase.	25
18.1	Dynamic header states based on available width.	32
19.1	Standard <code>biblatex</code> citation commands and descriptions.	33
19.2	BibLaTeX citation styles.	35
B.1	Submission metadata fields.	39
B.2	Title page metadata fields.	40

B.3	Bibliography metadata fields.	40
B.4	Font and spacing metadata fields.	41
B.5	Colour metadata fields.	41
B.6	Numbering metadata fields.	42
B.7	Minted metadata field.	42
D.1	General mathematical notation.	46
D.2	Algebra notation.	46
D.3	Physics notation.	47
E.1	Lowercase Greek letters.	48
E.2	Uppercase Greek letters (only those that differ from Latin capital letters).	48
F.1	Logic and set notation.	49
F.2	Relation symbols.	49
F.3	Arrow symbols.	49
G.1	Text font commands.	50
G.2	Font size commands.	50

List of Algorithms

12.1	Binary Search	23
12.2	Bubble Sort	24

List of Listings

17.1	MATLAB example.	30
17.2	Java example.	31
17.3	Python example.	31
17.4	C++ example.	31
17.5	LaTeX example.	31
A.1	Appendix Section Code.	37
A.2	Appendix Section Cross-Reference.	37

1 Conditionals & Utilities

1.1 xifthen

This package is used internally by the template and is not typically invoked directly by the user.

The `xifthen` package lets the template make simple yes/no decisions based on your metadata settings in `main.tex`. For example, it checks whether `\metaShowTodo` is set to `true` and shows or hides the todo list accordingly. You do not need to write any `xifthen` commands yourself, just set your metadata values and the template handles the rest.

1.2 xstring

This package is used internally by the template for the smart header logic. Users do not use it directly.

The `xstring` package gives the template tools to read and work with text strings. The template uses it to check the length of `\metaCourseName` and decide whether to show the full name or a shorter acronym in the header. The table below shows what `xstring` can do:

Table 1.1: Common `xstring` operations.

Operation	Command	Result
Extraction	<code>\StrLeft{Lassonde}{4}</code>	Lass
Substitution	<code>\StrSubstitute{12/05/26}{/}{-}</code>	12-05-26
Length	<code>\StrLen{YorkU}</code>	5
Counting	<code>\StrCount{Engineering}{n}</code>	3

1.3 lipsum

This package is used internally by the template for layout testing. It is not intended for use in final documents.

The `lipsum` package generates placeholder text for testing the layout and spacing of a page. Use `\lipsum[n]` to generate paragraph n , or `\lipsum[1-3]` to generate a range of paragraphs. The text is meaningless Latin and should be replaced with real content before submitting your document.

An example of a paragraph of generated text:

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero,

nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

2 Colours

2.1 xcolor

This package is used internally by the template for colour definitions and tints. Users may use it for custom colouring, but it is not required for normal document preparation.

The `xcolor` package adds colour support to $\text{\LaTeX}$. The template uses it to apply your chosen theme colour to headings, horizontal rules, boxes, and highlighted notes throughout the document.

2.1.1 Setting the Theme Colour

You set the theme colour by editing `\metaThemeColour` in `main.tex`. You can type a hex colour code, the name of any built-in colour, or the name of a custom colour defined in the template. If you leave it empty, the template uses `DefaultBlue`.

Table 2.1: Accepted values for `\metaThemeColour`.

Type	Example Value	Effect
Empty		Defaults to <code>DefaultBlue</code>
Hex code	<code>#810001</code>	Parsed as an HTML hex colour
Named colour	<code>red</code>	Resolved via <code>\colorlet</code>
Custom colour	<code>LassondeNavy</code>	Any colour defined in the sty file

The same applies to `\metaLinkColour`, `\metaCiteColour`, and `\metaUrlColour`; each one also falls back to `themecolour` when left empty.

2.1.2 Custom Defined Colours

These colours are built into the template and can be typed directly into `\metaThemeColour` or any colour command:

Table 2.2: Custom colours defined in `stemtemplate.sty`.

Name	Hex	Preview
DefaultBlue ^a	#1B4F8A	
YorkRedDark	#810001	
ScienceSkyBlueDark ^b	#065C87	
LassondeNavy ^c	#003366	

^a Unaffiliated professional blue.^b Recommended for York U Faculty of Science students.^c Recommended for York U Lassonde students.

2.1.3 Built-in Named Colours

You can also use any of `xcolor`'s built-in colour names. A small selection is shown below. For the full list, see <https://mirrors.ctan.org/macros/latex/contrib/xcolor/xcolor.pdf>:

Table 2.3: Built-in `xcolor` named colours.

Name	Preview	Name	Preview
<i>Base colours</i>			
red		cyan	
green		magenta	
blue		yellow	
black		white	
gray		brown	
<i>Extended colour sets (dvipsnames, svgnames, x11names)</i>			
NavyBlue		ForestGreen	
Maroon		BurntOrange	
RoyalPurple		Sepia	
SlateGray		Teal	
SteelBlue4		Firebrick3	

2.1.4 Theme Colour and Tints

Adding `!` followed by a number after any colour name creates a lighter version of that colour, called a *tint*. The number is the percentage of the original colour, so `themecolour!75` is 75% colour and 25% white.

You can extend this to mix with any second colour, not just white, by adding a second ! and colour name: `colourA!X!colourB` gives $X\%$ `colourA` blended with $(100 - X)\%$ `colourB`. Mixing toward black instead of white produces a *shade* rather than a tint, and mixing toward any other colour produces a custom blend:

Table 2.4: Active theme colour, tints, shades, and blends.

Name	Mix	Preview
<i>Tints – blended toward white</i>		
<code>themecolour</code>	100% colour	
<code>themecolour!75</code>	75% colour	
<code>themecolour!50</code>	50% colour	
<code>themecolour!25</code>	25% colour	
<i>Shades – blended toward black</i>		
<code>themecolour!100!black</code>	100% colour, 0% black	
<code>themecolour!75!black</code>	75% colour, 25% black	
<code>themecolour!50!black</code>	50% colour, 50% black	
<code>themecolour!25!black</code>	25% colour, 75% black	
<i>Blends – toward another colour</i>		
<code>themecolour!70!cyan</code>	70% colour, 30% cyan	
<code>themecolour!70!magenta</code>	70% colour, 30% magenta	
<code>themecolour!70!yellow</code>	70% colour, 30% yellow	
<code>themecolour!70!red</code>	70% colour, 30% red	
<code>themecolour!70!green</code>	70% colour, 30% green	
<code>themecolour!70!blue</code>	70% colour, 30% blue	

2.1.5 Using `\textcolor` and `\colorbox`

Use `\textcolor` to change the colour of a word or phrase in your text. It takes two arguments: the colour name and the text to colour. For example, `\textcolor{themecolour}{Sample Text}` colours the text without changing anything else on the line.

Use `\colorbox` to place a coloured background behind text, useful for drawing attention to a short note or label. For example, `\colorbox{themecolour!15}{Sample Text}` produces a lightly shaded highlight box.

3 Page Geometry

3.1 `geometry`

This package is used internally by the template to configure page margins and layout. Users do not normally modify these settings.

The `geometry` package controls the page margins and the size of the text area. The template uses it to set standard margins automatically, you do not need to touch anything for your document to be formatted correctly.

3.2 `setspace`

This package is used internally by the template to control line spacing. Users normally do not interact with it directly, but spacing can be adjusted through metadata commands in `main.tex`.

The `setspace` package controls the space between lines of text. You can choose between single, one-and-a-half, or double spacing by setting the appropriate metadata value in `main.tex`, you do not need to call any `setspace` commands yourself.

3.3 pdfscape

This package is available for rotating wide tables or figures into landscape orientation. It is not required for normal document preparation and is only used when the author explicitly includes a landscape environment.

The pdfscape package lets you rotate a page sideways so that wide content fits. Wrap anything in a landscape environment and that page will display in landscape orientation in the PDF viewer. The page after it returns to portrait automatically.

The table below is an example of content that is too wide for portrait and benefits from a landscape page:

Table 3.1: Wide data table spanning landscape page.

Parameter	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5
Temperature (°C)	23.4	24.1	22.8	23.9	24.5
Pressure (kPa)	101.3	101.1	101.5	100.9	101.4
Humidity (%)	48.7	50.2	47.9	51.0	49.3
Voltage (V)	4.98	5.01	4.97	5.02	5.00
Current (mA)	120.1	119.8	120.4	120.0	119.6

Use `\thispagestyle{plain}` on the line after `\begin{landscape}` to remove the header from the rotated page. Use `\thispagestyle{empty}` instead if you also want to hide the page number in the footer.

4 Language & Font

4.1 babel

This package manages language-specific typographic rules. The template loads it automatically with English as the active language.

The `babel` package tells $\text{\LaTeX}$ which language the document is written in. The template loads it with English so that words are hyphenated correctly and punctuation follows English conventions. No changes are needed on your end.

4.2 fontspec

This package provides system font access under XeLaTeX. It is loaded automatically by the template.

The `fontspec` package is the foundation of font loading under Xe $\text{\LaTeX}$. It allows the template to load any font installed on your system by name, and handles all character encoding internally. You do not need to configure it; the template applies it automatically based on your metadata.

4.2.1 `\metaSerifFont` and `\seriftext`

The serif font is configured through metadata. Users typically only use the `\seriftext{}` command directly.

The `\metaSerifFont` metadata key sets the serif font for the document. Leave it empty to use the default, IBM Plex Serif. To override it, set it to any font installed on your system by exact name, for example `Georgia` or `Times New Roman`.

Use `\seriftext{}` to apply the serif font to a specific word or phrase regardless of which font the document body is using:

This text is rendered in the serif font.

4.2.2 `\metaSansFont` and `\sansseriftext`

The sans-serif font is configured through metadata. Users normally interact with it only through `\sansseriftext{}`.

The `\metaSansFont` metadata key sets the sans-serif font for the document. Leave it empty to use the default, IBM Plex Sans. To override it, set it to any font installed on your system by exact name, for example `Arial` or `Helvetica Neue`.

Use `\sansseriftext{}` to apply the sans-serif font to a specific word or phrase:

This text is rendered in the sans-serif font.

4.2.3 Default Font

The template selects the global font through the metadata in `main.tex`; users do not modify font packages directly.

You control which font family the document body uses by setting `\metaDefaultFont` in `main.tex`. The current setting is `sans`. Set it to `sans` for the sans-serif font or `serif` for the serif font. The local commands `\seriftext{}` and `\sansseriftext{}` always work regardless of which global font is active.

To use a required font such as Arial for an assignment, set `\metaSansFont{Arial}` and `\metaDefaultFont{sans}`. The serif font remains available via `\seriftext{}` as a companion.

5 Spacing Around Headings

5.1 titlesec

This package is used by the template to control heading formatting and spacing. Users typically do not modify it directly.

The `titlesec` package controls how headings look throughout the document. The template uses it to make all headings bold and coloured with `themecolour`. You do not need to change anything here.

5.1.1 Heading Levels

$\text{\LaTeX}$ provides up to seven heading levels, though not every document class makes all of them available. The table below lists each level, whether it exists in `article` (used by this template), and an example of its appearance.

Table 5.1: Heading levels and their appearance in this template.

Level	Command	Example
-1	<code>\part^a</code>	Part I
		Heading Example
0	<code>\chapter^b</code>	<i>Not available</i>
1	<code>\section</code>	1 Heading Example
2	<code>\subsection</code>	1.1 Heading Example
3	<code>\subsubsection</code>	1.1.1 Heading Example

Level	Command	Example
4	<code>\paragraph</code>	Heading Example. Body text follows on the same line.
5	<code>\subparagraph</code>	Heading Example. Indented, body text follows on the same line.

^a Rarely used in assignment-style documents.

^b Not available in this template. Requires switching to report or book.

Note that `\paragraph` and `\subparagraph` are run-in headings by default, meaning the body text begins on the same line rather than on its own line below, as shown above. Also note that only `\section` through `\subsubsection` are numbered by default, `\paragraph` and `\subparagraph` do not display numbers.

5.1.2 Abstract environment

The abstract environment is customised by the template to ensure consistent formatting. Users typically write their abstract normally without modifying this configuration.

By default, $\text{\LaTeX}$ centres the abstract text in italic. The template overrides this so the abstract looks like a regular section heading instead, which is the bold flush-left “Abstract” heading you see at the start of this document. You write your abstract the same way either way, just inside `\begin{abstract}` and `\end{abstract}`.

6 Math

6.1 `amsmath`, `amssymb`, `amsfonts`

These three packages are the standard tools for writing math in $\text{\LaTeX}$. Together they give you display math environments, a large collection of math symbols, and blackboard bold letters for number sets like $\mathbb{R}$, $\mathbb{Z}$, and $\mathbb{N}$.

6.1.1 `equation`

The `equation` environment puts a single equation on its own line with a number on the right. Use `equation*` if you do not want the number:

$$2 + 2 = 4 \tag{6.1}$$

$$\boxed{a^2 + b^2 = c^2} \tag{6.2}$$

6.1.2 align

The `align` environment lets you write multiple equations that line up at the `&` character. Each line gets its own number unless you add `\nonumber` to that line:

$$\begin{aligned} y &= 2x + 1 \\ &= 2(3) + 1 \end{aligned} \tag{6.3}$$

6.1.3 gather

The `gather` environment puts each equation on its own centred line with no alignment between them. Use it when you have a group of separate equations to display together:

$$x + y = 5 \tag{6.4}$$

$$x - y = 1 \tag{6.5}$$

6.1.4 multiline

The `multiline` environment is for a single equation that is too long to fit on one line. The first line is left-aligned, the last line is right-aligned, and any lines in between are centred:

$$\begin{aligned} (a_1 + a_2 + a_3) \times (b_1 + b_2 + b_3) = \\ a_1b_1 + a_1b_2 + a_1b_3 + a_2b_1 + a_2b_2 + \\ a_2b_3 + a_3b_1 + a_3b_2 + a_3b_3 \end{aligned} \tag{6.6}$$

6.1.5 flalign

The `flalign` environment works like `align` but pushes the columns to the far left and far right of the page. It is useful when you want one equation flush left and another flush right on the same line:

$$F = ma \qquad E = mc^2 \tag{6.7}$$

6.1.6 alignat

The `alignat` environment is similar to `align` but lets you control the spacing between columns manually. The required argument is the number of column pairs. Use it when `align` produces too much or too little space between groups:

$$x = 3 \quad y = 7 \quad (6.8)$$

$$x = -1 \quad y = 2 \quad (6.9)$$

6.1.7 split

The `split` environment breaks one long equation across multiple lines while keeping a single equation number. It must be placed inside `equation`:

$$\begin{aligned} (a + b)^2 &= a^2 + 2ab \\ &\quad + b^2 \end{aligned} \quad (6.10)$$

6.1.8 cases

The `cases` environment is used for piecewise functions. It draws an opening brace and lets you write a different expression for each condition:

$$|x| = \begin{cases} x & \text{if } x \geq 0 \\ -x & \text{if } x < 0 \end{cases} \quad (6.11)$$

6.2 mathtools

The `mathtools` package adds a few extra tools on top of `amsmath`. The most useful for everyday writing is `\coloneqq`, which produces the “defined as equal to” symbol:

$$f(x) \coloneqq 2x + 3 \quad (6.12)$$

It also provides matrix environments with different bracket styles. Each one works the same way but uses different surrounding symbols:

$$\text{pmatrix: } \begin{pmatrix} a & b \\ c & d \end{pmatrix} \quad \text{bmatrix: } \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad \text{vmatrix: } \begin{vmatrix} a & b \\ c & d \end{vmatrix} \quad \text{Vmatrix: } \begin{Vmatrix} a & b \\ c & d \end{Vmatrix} \quad (6.13)$$

6.3 `bm`

The `bm` package gives you the `\bm{}` command, which makes a math symbol bold while keeping it italic. This is a common way to write vectors and matrices in science and engineering, since `\mathbf` would make the letter upright:

$$\boldsymbol{F} = m\boldsymbol{a} \quad \boldsymbol{v} = \boldsymbol{u} + \boldsymbol{a}t \quad (6.14)$$

6.4 `siunitx`

The `siunitx` package formats numbers and units correctly. Use `\SI{value}{unit}` when you have a number and a unit together, and `\si{unit}` when you just need the unit on its own.

Table 6.1: SI unit examples using `siunitx`.

Command	Output
<code>\SI{9.81}{\metre\per\second\squared}</code>	9.81 m/s ²
<code>\SI{3e8}{\metre\per\second}</code>	3 × 10 ⁸ m/s
<code>\SI{100}{\kilo\gram}</code>	100 kg
<code>\SI{1.6e-19}{\coulomb}</code>	1.6 × 10 ⁻¹⁹ C
<code>\si{\ohm}</code>	Ω
<code>\si{\kilo\pascal}</code>	kPa

Units can also be used inline inside a sentence. For example, gravitational acceleration is 9.81 m/s², and standard atmospheric pressure is 101.325 kPa.

7 Chemistry

7.1 `mhchem`

The `mhchem` package lets you write chemical formulas and reactions using the `\ce{}` command. You just type the formula naturally and it handles all the subscripts, superscripts, and charges for you.

You can use it inline inside a sentence: water is H₂O, table salt is NaCl, and the sulfate ion is SO₄²⁻.

7.2 Reaction Environments

This template provides six numbered reaction environments, one for each math display environment. All of them use a separate reaction counter that is independent of

equation numbering, so reactions are labelled R1, R2, R3 and equations are labelled separately. When section numbering is enabled via `\metaNumberEquations`, reactions are also numbered by section in the same way.

If you prefer reactions to be numbered sequentially as 1, 2, 3... rather than R1, R2, R3..., simply use the standard math display environments instead of the dedicated reaction environments; in that case, reactions will share the same counter as regular equations.

Use `\label{}` and `\cref{}` to cross-reference reactions exactly as you would an equation.

7.2.1 `requation`

The `requation` environment is the reaction equivalent of `equation`. Use it for a single reaction on its own line:



Adding `->[\Delta]` puts a delta above the arrow, commonly used to show that heat or a catalyst is applied:



7.2.2 `ralign`

The `ralign` environment is the reaction equivalent of `align`. Use it to show a multi-step mechanism where each step lines up at the `&` character:



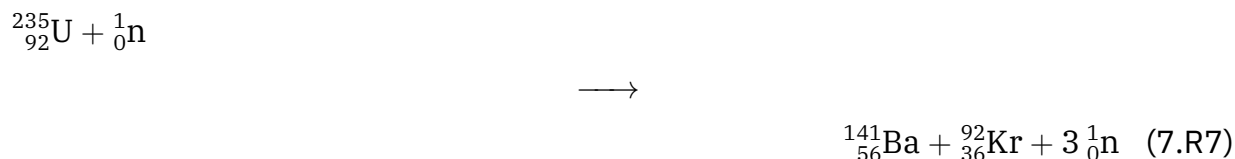
7.2.3 `rgather`

The `rgather` environment is the reaction equivalent of `gather`. Use it for a group of related reactions displayed together with no column alignment:



7.2.4 `rmultline`

The `rmultline` environment is the reaction equivalent of `multline`. Use it when a single reaction is too long to fit on one line. The first line is left-aligned, the last line is right-aligned, and any lines in between are centred:



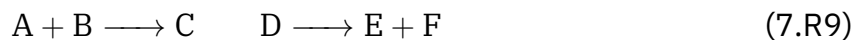
7.2.5 `rflalign`

The `rflalign` environment is the reaction equivalent of `flalign`. It pushes columns to the far left and far right of the page, useful when you want to display two reactions side by side at opposite ends of the line:



7.2.6 `ralignat`

The `ralignat` environment is the reaction equivalent of `alignat`. The required argument is the number of column pairs. Use it when you need precise manual control over the spacing between columns:



7.2.7 `split`

The `split` environment works exactly the same here as it does in standard math: it breaks a long reaction across multiple lines while keeping a single number. The `split` environment itself does not produce an “R” label, so it must be placed inside `requeation` if you want the reaction to receive an R-style number:

$$\begin{array}{l} A + B \longrightarrow C + D \\ \longrightarrow E \end{array} \quad (7.R10)$$

8 Theorem Environments

8.1 amsthm

The `amsthm` package gives you styled boxes for presenting formal mathematical statements. The template defines three visual styles: `plain`, `definition`, and `remark`. Each style is used by a different group of environments shown below.

8.1.1 Plain — Theorem, Lemma, Proposition, Corollary, Conjecture, Axiom, Proof

Theorem 8.1: *For any right triangle with sides a and b and hypotenuse c : $a^2 + b^2 = c^2$.*

Proof: Draw a square on each side of the triangle and compare the areas. ■

The end-of-proof symbol `\qed` is redefined in `stemtemplate.sty` to show a solid `themecolour` square (■) instead of the default hollow box.

Lemma 8.1: *If n is an even number, then n^2 is also even.*

Proposition 8.1: *The sum of two whole-number fractions is always a whole-number fraction.*

Corollary 8.1: *If p is a prime number greater than 2, then p is odd.*

Conjecture 8.1: *Every even number greater than 2 can be written as the sum of two prime numbers.*

Axiom 8.1: *Through any two points there is exactly one straight line.*

8.1.2 Definition — Definition, Example, Criterion

Definition 8.1: An even number is any integer that can be divided by 2 with no remainder.

Example 8.1: The numbers 2, 4, 6, and 8 are all even numbers.

Criterion 8.1: A number is positive if and only if it is greater than zero.

8.1.3 Remark — Remark, Note

Remark 8.1: Zero is even because it can be divided by 2 with no remainder.

Note 8.1: All theorem environments are automatically numbered by section when `\metaNumberTheorems` is set to `true`.

9 Figures

9.1 `graphicx`

The `graphicx` package provides the `\includegraphics` command for placing images in your document. The template is set up to look for images in the `figures/` and `assets/` folders, so you only need to type the filename with no folder path.

You can control the size of the image with optional arguments: `width=0.6\linewidth` scales it to 60% of the text width, `height=5cm` sets a fixed height, and `scale=0.5` makes it half its original size. You only need to set one of these and the other dimension scales automatically to keep the proportions correct.



Figure 9.1: Faculty logo included with `\includegraphics`.

9.2 `float`

This package is used internally by the template to enable the `[H]` placement specifier. Users interact with it by adding `[H]` to any `figure` or `table` environment.

9.3 `subcaption`

The `subcaption` package lets you place two or more related figures or tables side by side under a shared caption. Each one gets its own label and caption, and the group gets a shared parent label. They are lettered automatically as (a), (b), (c), and so on.



Figure 9.2: Two sub-figures using `subcaption`.

(a) First sub-figure.

(b) Second sub-figure.

9.4 `caption`

This package is used internally by the template to define caption formatting. Users do not modify it directly; captions are written normally with `\caption{}`.

The caption package controls how figure and table captions look. The template uses it to make the label bold and coloured in `themecolour`, followed by a colon, with the caption text left-aligned in a slightly smaller font. This applies automatically to every `\caption` in your document with no extra work needed.

9.5 pdfpages

The pdfpages package lets you insert pages from another PDF directly into your document using `\includepdf[options]{filename.pdf}`. By default the inserted pages have no header or footer. Pass `pagecommand={\thispagestyle{fancy}}` to bring them back.

Table 9.1: Common `\includepdf` options.

Option	Effect
<code>pages=-</code>	Include all pages
<code>pages={1,3}</code>	Include pages 1 and 3 only
<code>pages={2-5}</code>	Include a page range
<code>landscape</code>	Rotate inserted pages to landscape
<code>nup=2x1</code>	Place two source pages side by side on one sheet
<code>pagecommand={\thispagestyle{fancy}}</code>	Restore header and footer on inserted pages

10 Tables

10.1 booktabs

The booktabs package gives you three horizontal line commands for clean, professional tables: `\toprule` at the top, `\midrule` between the header row and the data, and `\bottomrule` at the bottom. Always use these instead of `\hline`.

Table 10.1: Basic table using booktabs.

Parameter	Value	Unit
Resistance	10	Ω
Current	2	A
Voltage	20	V

10.2 array

This package is used internally by the template and is required by `tabularx` and `multirow`. Users may use its column specifiers directly when fine-grained column control is needed.

The `array` package lets you apply formatting to an entire column at once. The `>...` syntax goes before the column letter in your column definition and applies a command to every cell in that column. The example below uses it to centre and wrap text in two fixed-width columns. The template defines shorthand column types `P{}`, `Q{}`, and `K{}` as `ragged2e`-aware equivalents of `p{}` for left, centre, and right alignment respectively:

Table 10.2: Fixed-width centred columns using `array`.

Left column	Right column
Centred text	Centred text
Long cell content wraps	Also wraps cleanly

Note: Example of a table note.

10.3 tabularx

`tabularx` is used internally by `ltablex` and is not the preferred table environment in this template. Use `ltablex` for most tables. `tabularx` is only recommended when required to have a table anchored to the top or bottom of a page.

The `tabularx` package adds the `X` column type, which automatically stretches to fill the remaining width of the table. You set the total table width as the first argument (usually `\linewidth`) and any `X` columns share the leftover space equally.

The template defines three `ragged2e`-aware column types built on `X`: `L` (ragged right), `C` (centred), and `R` (ragged left). These should be preferred over plain `X` for columns containing wrapped text, as they allow hyphenation and produce more even line lengths.

`tabularx` is, however, the preferred environment for a table that must be anchored to the top or bottom of a page, since `ltablex` is not a true float and cannot accept a placement specifier such as `[t]` or `[b]`. For this case, wrap `tabularx` in a real table float. Doing so causes the shared `table` counter to silently increment twice per table; add `\addtocounter{table}{-1}` immediately after the `tabularx` environment, inside the float, to correct it:

Table 10.3: Common float placement specifiers.

Specifier	Meaning
t	Places the float at the top of the current or next page.
b	Places the float at the bottom of the current or next page.
H	Forces the float to appear exactly where it is written in the source, with no floating at all. Requires the <code>float</code> package.

Note: Example of a table note.

10.4 `multirow`

The `multirow` package lets a single cell span more than one row. The command is `\multirow{n}{width}{content}`, where `n` is the number of rows to span and `width` is usually `*` to let $\text{\LaTeX}$ set it automatically. Leave the cells below the merged cell empty. Adding a `\midrule` between groups helps separate them visually:

Table 10.4: Table with multi-row cells using `multirow`.

Category	Parameter	Value
Electrical	Resistance	10 Ω
	Current	2 A
Mechanical	Force	50 N
	Velocity	3 m/s

10.5 `ltablex`

The `ltablex` package extends `tabularx` to support page breaks. It automatically handles tables that are too long to fit on one page, repeating the header on each new page. The X-based column types (L, C, R) work as normal. The `\endfirsthead`, `\endhead`, and `\endfoot` commands define the repeating header and closing rule. Always use `tabularx`, never `longtable` directly:

Table 10.5: Table packages and their purposes, demonstrating `ltablex` with an L column.

Package	Purpose
<code>booktabs</code> ^a	Professional horizontal rules (<code>\toprule</code> , <code>\midrule</code> , <code>\bottomrule</code>)

Package	Purpose
<code>array</code> ^b	Extended column formatting; required by <code>tabularx</code> and <code>multirow</code>
<code>tabularx</code> ^c	Flexible X columns that stretch to fill the table width
<code>multirow</code> ^d	Cells that span more than one row
<code>ltablex</code> ^e	Extends <code>tabularx</code> with page-breaking; always use <code>tabularx</code> , never <code>longtable</code> directly

Note: Example of a table note.

^a Note on `booktabs`.

^b Note on `array`.

^c Note on `tabularx`.

^d Note on `multirow`.

^e Note on `ltablex`.

11 Diagrams & Plots

11.1 `tikz`

The `tikz` package lets you draw diagrams directly in your document using simple drawing commands. Everything is described with coordinates and paths in code, so diagrams scale perfectly and can include $\text{\LaTeX}$ math and text naturally.

The basic drawing command is `\draw[options] (start) -- (end);`. The `->` option adds an arrowhead and `rectangle` draws a box between two corner coordinates. Coordinates are written as (x,y) in centimetres by default.

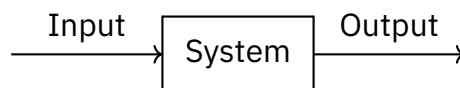


Figure 11.1: Simple block diagram drawn with `tikz`.

11.2 `pgfplots`

The `pgfplots` package builds data plots on top of `tikz`. Plots go inside an `axis` environment, which handles the axis labels, tick marks, grid lines, and scaling for you. Use `\addplot{expression}` to plot a mathematical function, or supply explicit coordinates to plot data points.

Useful axis options include `width`, `height`, `xlabel`, `ylabel`, `xmin/xmax`, `ymin/ymax`, and `grid`. The `domain` option on `\addplot` sets the range of x values and `samples` controls how

smooth the curve looks.

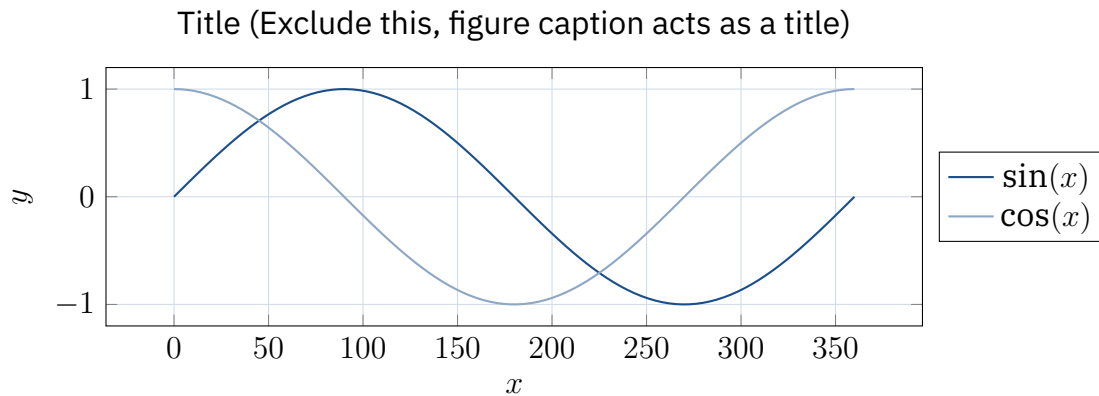


Figure 11.2: Sine and cosine waves plotted with pgfplots.

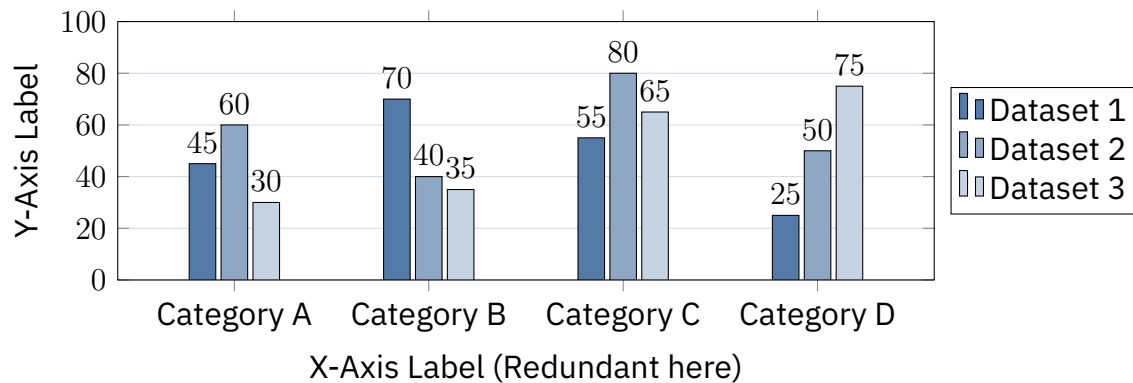


Figure 11.3: Bar graph plotted with pgfplots.

11.3 circuitikz

The `circuitikz` package extends `tikz` with circuit symbols for drawing electrical schematics. Components are placed along a path using `to[component, label]`, where the component name is a standard symbol such as `R` (resistor), `C` (capacitor), `L` (inductor), `V` (voltage source), or `I` (current source). Paths are chained together to form a closed loop.

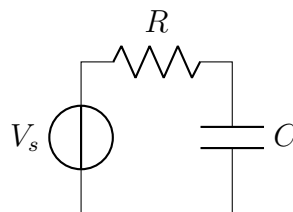


Figure 11.4: Simple RC circuit drawn with `circuitikz`.

11.4 pgf-pie

The `pgf-pie` package draws pie charts directly inside a `tikzpicture` environment. Each slice is specified as a percentage followed by a label, separated by a forward slash, and the values are listed inside the `\pie` command. The package automatically scales the slices so that the percentages add up to a full circle.

Useful options include `radius` to control the size of the chart, `text=legend` to place labels in a legend below instead of on the slices, `text=pin` to connect labels with lines, and `color` to supply a list of slice colours. Using `themecolour` with tints keeps the chart consistent with the rest of the document.

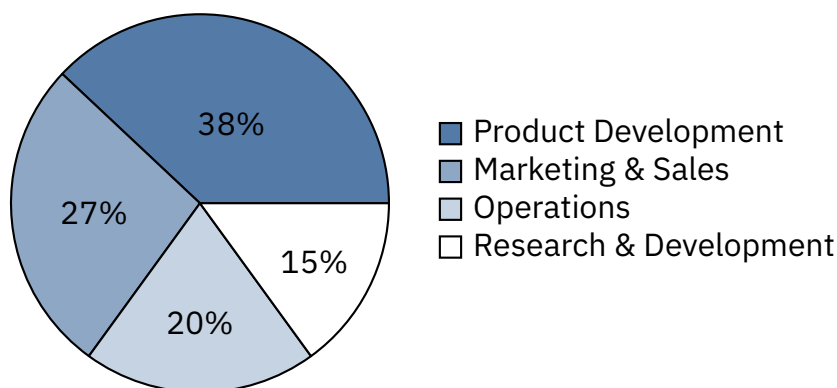


Figure 11.5: Annual budget allocation across departments.

11.5 environ

This package is used internally by the template for draft-mode figure suppression. Users do not use it directly.

The `environ` package lets the template capture the contents of a `tikzpicture` or `circuitikz` environment without running them. When `\metaShowFigures` is `false`, this is what allows vector graphics to be replaced with a lightweight placeholder box instead of being fully computed and drawn, which speeds up compiling while drafting.

12 Algorithms

12.1 algorithm and algpseudocode

The `algorithm` package provides a float wrapper for pseudocode, giving algorithms captions, labels, and an entry in a list of algorithms, the same way `figure` wraps a `tikzpicture`. The `algpseudocode` package supplies the pseudocode commands that go inside the `algorithmic` environment.

The algorithmic environment takes an optional line-numbering argument: [1] numbers every line, [2] numbers every second line, and omitting it disables numbering entirely.

Table 12.1: Common algpseudocode commands.

Command	Purpose
<code>\Require</code>	Declares what the algorithm expects as input
<code>\Ensure</code>	Declares what the algorithm produces as output
<code>\State</code>	A single statement on its own line
<code>\Function{name}{params}</code>	Opens a named function block
<code>\If / \ElsIf / \Else / \EndIf</code>	Conditional branch
<code>\For / \EndFor</code>	For loop
<code>\While / \EndWhile</code>	While loop
<code>\Return</code>	Return statement
<code>\Comment{...}</code>	Inline comment, right-aligned

The two algorithms below demonstrate a function with nested control flow. Indentation is handled automatically with no manual spacing required:

Algorithm 12.1: Binary Search

Input: Sorted array A of length n , search value $target$

Output: Index of $target$ in A , or -1 if not found

```

1: function BinarySearch( $A, n, target$ )
2:    $low \leftarrow 0$ 
3:    $high \leftarrow n - 1$ 
4:   while  $low \leq high$  do
5:      $mid \leftarrow \lfloor (low + high)/2 \rfloor$                                 ▷ Compute midpoint
6:     if  $A[mid] = target$  then
7:       return  $mid$                                                     ▷ Target found
8:     else if  $A[mid] < target$  then
9:        $low \leftarrow mid + 1$                                           ▷ Search right half
10:    else
11:       $high \leftarrow mid - 1$                                           ▷ Search left half
12:    end if
13:  end while
14:  return  $-1$                                                           ▷ Target not found
15: end function

```

Algorithm 12.2: Bubble Sort**Input:** Array A of length n **Output:** A sorted in ascending order

```

1: function BubbleSort( $A, n$ )
2:   for  $i \leftarrow 0$  to  $n - 1$  do
3:     for  $j \leftarrow 0$  to  $n - i - 2$  do
4:       if  $A[j] > A[j + 1]$  then
5:         swap  $A[j]$  and  $A[j + 1]$ 
6:       end if
7:     end for
8:   end for
9: end function

```

▷ Swap out-of-order elements

Algorithms are referenced using `\cref` like any other float. algorithm 12.1 searches a sorted list by repeatedly halving the search range, while algorithm 12.2 sorts a list by repeatedly swapping neighbouring elements that are in the wrong order.

13 Links & References

13.1 hyperref

This package is used internally by the template to configure clickable links and PDF meta-data. Users interact with it through `\url{}` and `\href{}{}` for external links; all internal cross-references are handled automatically.

The `hyperref` package makes every internal cross-reference, citation, and table of contents entry clickable in the PDF. It also embeds document information such as the title and author into the PDF file. The template sets all link colours to `themecolour` automatically.

The two commands you will use directly are `\url{}` for a plain clickable link and `\href{url}{text}` for a link with custom display text:

- `\url{https://www.yorku.ca}` → <https://www.yorku.ca>
- `\href{https://www.yorku.ca}{York University}` → [York University](https://www.yorku.ca)

13.2 cleveref

This package is used internally by the template to configure reference formatting. Users interact with it directly through `\cref{}` and `\Cref{}` for all cross-references.

The `cleveref` package replaces the standard `\ref` command with `\cref`, which automatically adds the correct label type in front of the number. For example, `\cref{fig:logo}` produces fig. 9.1 without you having to type the word “Figure” yourself. Use `\Cref` (capital C) at the start of a sentence. You can also pass multiple labels separated by commas and `cleveref` will group them neatly.

Table 13.1: Cross-reference examples using `\cref` for lowercase and `\Cref` for uppercase.

Type	Output
<code>\cref</code>	
Figure	fig. 9.1
Sub-figure	fig. 9.2a
Table	table 10.1
Equation	eq. (6.1)
Reaction	rxn. (7.R1)
Algorithm	algorithm 12.1
Section	section 15
Appendix	appendix A
Theorem	theorem 8.1
Lemma	lemma 8.1
Proposition	proposition 8.1
Corollary	corollary 8.1
Conjecture	conjecture 8.1
Axiom	axiom 8.1
Definition	definition 8.1
Example	example 8.1
Criterion	criterion 8.1
Remark	remark 8.1
Note	note 8.1
Problem	problem 16.1
Listing	listing 17.1
<code>\Cref</code>	
Figure	Fig. 9.1
Sub-figure	Fig. 9.2a
Table	Table 10.1
Equation	Eq. (6.1)

Type	Output
Reaction	Rxn. (7.R1) <i>etc...</i>

14 Typography

14.1 `microtype`

This package is used internally by the template to improve text rendering. It requires no user configuration and produces no visually distinct output on its own.

The `microtype` package makes small automatic adjustments to character spacing and sizing to produce cleaner, more even-looking paragraphs. The result is fewer lines that stick out into the margin and less awkward hyphenation, particularly in dense paragraphs with long words. It works silently in the background with no action needed from you.

14.2 `xurl`

This package is used internally by the template to improve URL line breaking. Users interact with it through `\url{}`, which is provided by `hyperref`.

The `xurl` package allows long URLs to break across lines at sensible points such as slashes, dots, and hyphens. Without it, a long URL is treated as one unbreakable word and can stick out past the margin. The URL below is long enough to demonstrate a clean line break:

<https://futurestudents.yorku.ca/program/engineering-intl-development-studies>

14.3 `indentfirst`

This package is used internally by the template to enforce consistent paragraph indentation. It requires no user configuration.

By default, $\text{\LaTeX}$ does not indent the first paragraph of a section. The `indentfirst` package changes this so every paragraph is indented, including the first one. The paragraph below is the first in this subsection and is indented because `indentfirst` is loaded:

This is the first paragraph of this subsection, it is indented because `indentfirst` is loaded.

This is the second paragraph, also indented as normal.

14.4 ragged2e

This package is used internally by the template to improve text alignment in captions, boxes, and tables. Users interact with it through the column types L, C, R, P{ }, Q{ }, and S{ } defined in the template.

The ragged2e package provides improved versions of $\text{\LaTeX}$'s built-in alignment commands. The standard `\raggedright`, `\centering`, and `\raggedleft` commands disable hyphenation entirely, which can produce very uneven line lengths. The ragged2e versions (`\RaggedRight`, `\Centering`, and `\RaggedLeft`) allow hyphenation while still keeping text ragged, resulting in much more even-looking paragraphs. The template applies these automatically to captions, problem and solution boxes, and the custom table column types.

15 Lists

15.1 enumitem

The enumitem package gives you extra control over the spacing, labels, and indentation of $\text{\LaTeX}$'s three built-in list environments: `itemize`, `enumerate`, and `description`. Options are passed in square brackets directly on the environment.

The most useful option is `noitemsep`, which removes the extra vertical gap that $\text{\LaTeX}$ normally adds between items. Use it when list items are short and you want a tighter look:

- First item
- Second item
- Third item

The `label` option lets you change the bullet or number style. Use `\arabic*` for numbers, `\alph*` for lowercase letters, `\Alph*` for uppercase letters, `\roman*` for lowercase roman numerals, and `\Roman*` for uppercase roman numerals. The `*` is replaced by the item number automatically:

1. First item
2. Second item
3. Third item

- (a) First item
- (b) Second item
- (c) Third item

The `description` environment is useful for glossaries and key-value lists where each item has a bold label followed by a short explanation. The `leftmargin` option controls how far the body text is indented, and `style=nextline` places the body on a new line below the label, which helps when labels are long:

<code>noitemsep</code>	Removes vertical space between items and between the list and surrounding text
<code>label</code>	Overrides the default bullet or numbering style with any arbitrary format
<code>leftmargin</code>	Controls the left indent of the list body relative to the page margin
<code>style=nextline</code>	Places the description body on a new line below the label

Lists can be nested up to three levels deep. Indentation and bullet styles adjust automatically at each level:

- First level item
 - Second level item
 - * Third level item
- Another first level item

16 Problem & Solution Environments

16.1 `tcolorbox`

This package is used internally by the template to define the `problem` and `solution` environments. Users interact with it through those environments directly; the underlying `tcolorbox` configuration requires no modification.

The `tcolorbox` package provides the `problem` and `solution` environments. The `problem` environment is numbered automatically. When `\metaNumberProblems` is set to `true` in `main.tex`, problems are numbered by section (e.g. Problem 2.1, Problem 2.2). When set to `false`, problems are numbered from 1 continuously through the whole document. An optional title can be added in square brackets after `\begin{problem}` and it will appear in the header bar next to the number.

Problem 16.1: Ohm's Law

Given a resistor $R = 10\ \Omega$ with a voltage $V = 5\ \text{V}$ across it, find the current I .

Solution

Applying Ohm's Law:

$$I = \frac{V}{R} = \frac{5}{10} = 0.5\ \text{A} \quad (16.1)$$



The title argument is optional. When omitted, only the problem number appears in the header:

Problem 16.2

A problem without a title is also valid.

17 Code Listings

Every code block in this template, whether highlighted by `listings` or `minted`, is captioned and numbered off the same `lstlisting` counter. Place a `\captionof{lstlisting}{...}` and `\label{...}` directly above the code, and reference it anywhere with `\cref/\Cref` as usual. Because both engines write to the same counter, `listings` and `minted` blocks number continuously through one shared sequence no matter which produced a given block.

17.1 listings

`listings` is a drafting-speed fallback: it is what powers the `codelisting` environment whenever `\metaUseMinted` is false. Since it compiles natively (no external calls), it is fast, but its highlighting is less accurate and covers fewer languages than `minted`. Keep it on while drafting, and switch to `minted` for a final compile.

The `listings` package displays source code with syntax highlighting and line numbers. Five named styles are built into this template: `matlab`, `java`, `python`, `c++`, and `latex`. All styles share the same base look: a monospaced font, line numbers on the left in `themecolour`, a light grey background, and automatic line wrapping. Select a style by adding `style=name` as an option on the `lstlisting` environment.

For short inline code snippets, use `\lstinline[style=name]|...|` with `|` as the de-

limiter on both sides.

17.2 `minted`

This package is optional and controlled by `\metaUseMinted` in `main.tex`. It requires the `-8bit -shell-escape` compiler flag and a working Python/Pygments install.

`minted` is the **preferred** engine for code listings in this template, used automatically by the `codelisting` environment whenever `\metaUseMinted` is `true`. It uses the Python library Pygments to perform syntax highlighting instead of $\text{\LaTeX}$ macros, producing more accurate, professional-looking highlighting for a much wider range of languages than `listings` can match.

This accuracy comes at a real cost: `minted` shells out to Pygments on every compile, which is noticeably **slower** than `listings`' native, in- $\text{\LaTeX}$ highlighting. For that reason, it is recommended to leave `\metaUseMinted` set to `false` while actively drafting, and only switch it to `true` for a final, polished compile.

To use `minted`, compile with the `-8bit -shell-escape` flag enabled, and ensure Python and Pygments are installed and available on your system's PATH.

Use `\mintinline{language}{code}` for a short inline snippet:

```
public class HelloWorld {}
```

Unlike `\lstinline`, the block form of `minted` cannot be placed inside a table cell or a `tcolorbox` (including this template's `problem` and `solution` environments), since both rely on verbatim reading that conflicts with those contexts. Use `\mintinline{}{}` in those situations instead.

17.3 `codelisting` Environments

`codelisting` is this template's unified code environment. It automatically renders using `minted` or `listings`, whichever is currently active per `\metaUseMinted`, so every code block in a document uses one consistent syntax regardless of which engine produces it.

Write `\begin{codelisting}{style}...\end{codelisting}`, where `style` matches one of the named styles defined in `stemtemplate.sty` (`matlab`, `java`, `python`, `cpp`, `latex`). The same style name is used whether `minted` is on or off.

17.3.1 MATLAB

Listing 17.1: MATLAB example.

```
1 x = 0:0.1:2*pi; % This is a comment
2 y = sin(x);
3 plot(x, y);
```

```
4 xlabel('x'); ylabel('sin(x)');  
5 title('Sine Wave');
```

17.3.2 Java

Listing 17.2: Java example.

```
1 public class HelloWorld {  
2     public static void main(String[] args) {  
3         System.out.println("Hello, World!");  
4     } // This is a comment  
5 }
```

17.3.3 Python

Listing 17.3: Python example.

```
1 import numpy as np  
2 import matplotlib.pyplot as plt  
3  
4 x = np.linspace(0, 2 * np.pi, 100)  
5 y = np.sin(x)  
6 plt.plot(x, y)  
7 plt.show()
```

17.3.4 C++

Listing 17.4: C++ example.

```
1 #include <iostream>  
2 using namespace std;  
3  
4 int main() {  
5     cout << "Hello, World!" << endl;  
6     return 0;  
7 }
```

17.3.5 LaTeX

Listing 17.5: LaTeX example.

```
1 \begin{equation} % This is a comment  
2     E = mc^2  
3     \label{eq:einstein}  
4 \end{equation}  
5 See \cref{eq:einstein} for Einstein's mass-energy equivalence.
```


18 Header & Footer

18.1 fancyhdr

This package is used internally by the template to configure the page header and footer. Users control the content through metadata fields in `main.tex`; the `fancyhdr` configuration itself requires no modification.

The `fancyhdr` package gives you full control over the header and footer on every page. It divides both the header and footer into three zones: left, centre, and right. Each zone is set with its own command: `\lhead`, `\chead`, and `\rhead` for the header, and `\lfoot`, `\cfoot`, and `\rfoot` for the footer. The template fills these zones automatically using your metadata, so you do not need to call any of these commands yourself.

The decorative rule below the header is coloured in `themecolour` and its thickness is set to 0.8pt by the template. You can also define different headers for different sections of the document or for odd and even pages if needed.

18.1.1 Smart Collision Logic

The template includes a custom `\smartleftheader` command that automatically adjusts the left side of the header to prevent it from overlapping with the right side. It does this by measuring the available horizontal space and choosing one of four display modes:

1. **Full Mode:** Shows the course code followed by the full course name.
2. **Acronym Mode:** If the full name is too long, it is shortened to an acronym (e.g., “Object-Oriented Programming & Telemetry” becomes “OOP&T”).
3. **Minimal Mode:** If even the acronym is too long, only the course code is shown.
4. **Hidden Mode:** If the right side takes up nearly the full width, the left header is hidden completely to keep the header readable.

Table 18.1: Dynamic header states based on available width.

Scenario	Left Header Output
Not Crowded	Course Code: Course Name
A Little Crowded	Course Code: CN
Crowded	Course Code
Overcrowded	(Hidden)

The acronym is generated by the `\makeAcronym` command, which takes the first letter

of each significant word in `\metaCourseName` and skips short words like “the”, “of”, and “to”. See table 1.1 for the string commands it uses internally.

19 Bibliography

19.1 csquotes

This package is used internally by the template and is required by `biblatex`. It produces no visible output and requires no user configuration.

The `csquotes` package is a behind-the-scenes dependency of `biblatex` that ensures quotation marks in bibliography entries are typeset correctly for the active language. You can also use it directly in your document with `\enquote{}` to wrap a phrase in the correct quotation marks for English: “This is a quoted phrase.”

19.2 biblatex

The `biblatex` package handles all citation formatting and bibliography generation. References are stored in a `.bib` file, cited in the document with `\autocite{key}` (or one of several style-specific variants), and the full bibliography is printed at the end with `\printbibliography`.

Table 19.1 summarizes the available citation commands, and table 19.2 lists the supported citation styles. The active style is set via `\metaBibStyle` in `main.tex`; the style currently active in this document is `ieee`.

Table 19.1: Standard `biblatex` citation commands and descriptions.

Command	Output Format	Description
<code>\autocite{key}</code>	High-level context	Recommended default. Automatically adapts to the active style (parenthetical, superscript, or footnote).
<code>\cite{key}</code>	Bare citation	Basic citation output without extra surrounding parentheses or brackets (unless defined by style).
<code>\parencite{key}</code>	Parenthetical	Encloses the citation in brackets or parentheses, e.g., [1] or (Smith, 2026).

Command	Output Format	Description
<code>\textcite{key}</code>	Narrative / In-text	Integrates author name into sentence prose, e.g., Smith [1] or Smith (2026).
<code>\supercite{key}</code>	Superscript	Forces citation as a superscript number, e.g., ¹ or ^{1,2} .
<code>\footcite{key}</code>	Footnote	Places full or abbreviated citation in a footnote at the bottom of the page.
<code>\citeauthor{key}</code>	Author name(s) only	Prints only author names without year or index, e.g., Smith et al.
<code>\citeyear{key}</code>	Publication year only	Prints only the publication year, e.g., 2026.
<code>\citetitle{key}</code>	Title string only	Prints only the title of the entry, e.g., Thermal Control Design.
<code>\citeurl{key}</code>	URL link	Extracts and prints only the URL field from the entry.
<code>\nocite{key}</code>	Invisible	Includes the entry in the final bibliography without generating in-text output. Pass * to print all items.
<code>\cite[2]{key}</code>	Page	Passes a specific page or section number, e.g., [1, p. 2] or (Smith, 2026, p. 2).
<code>\cite[2-3]{key}</code>	Pages	Passes a specific page or section number, e.g., [1, pp. 2-3] or (Smith, 2026, pp. 2-3).
<code>[\bibstring{paragraph} 4]</code>	Paragraph	Suppresses default p. and inserts par., e.g., [1, par. 4].

Table 19.2: BibLaTeX citation styles.

Input	Name	Field	Example Output
chem-acs	ACS	Chemistry	1
phys	AIP	Physics / Astronomy	1
nature	Nature	Multidisciplinary Sci.	1
science	Science	Multidisciplinary Sci.	(1)
ieee	IEEE	Engineering	[1] or [1–3]
mla	MLA	Literature / Arts	(Smith 42)
apa	APA	Psychology / Education	(Smith, 2026)

After adding new references to the .bib file, BibTeX must be run once between compilations for new citations to resolve. Most editors, such as Overleaf and TeXstudio, handle this automatically when configured to use BibTeX as the bibliography backend.

For example, `\autocite[1–2]{example}` produces: [1, pp. 1–2]

References

- [1] J. Zhang, K. Fu, J. Qiu, X. Xia, H. Nie, and X. Wen, "Vibration characteristics of CFRP sandwich panels with acoustic black hole structures," *Mechanics of Advanced Materials and Structures*, vol. 33, no. 1, p. 2 445 223, Dec. 26, 2024, issn: 1537-6494. doi: [10.1080/15376494.2024.2445223](https://doi.org/10.1080/15376494.2024.2445223) Accessed: Apr. 26, 2026. [Online]. Available: <https://www.tandfonline.com/doi/full/10.1080/15376494.2024.2445223> (cit. on p. 35).

Appendix A Appendices

Place appendices at the end of your document after the bibliography. The `\appendix` command switches section numbering from arabic numbers to uppercase letters, so each `\section{}` after it becomes a new appendix: Appendix A, Appendix B, and so on. You do not need any extra configuration, just keep adding `\section{}` commands.

Listing A.1: Appendix Section Code.

```

1 \appendix
2 \section{First Appendix} % Appendix A
3 ...
4 \section{Second Appendix} % Appendix B
5 ...

```

A.1 Counter Resets

The template automatically resets the numbering of all major floats at the start of the appendix so they are labelled relative to their appendix letter. For example, the first figure in Appendix B is labelled **Figure B.1**. The following counters are reset automatically:

- figure → A.1, A.2, ...
- table → A.1, A.2, ...
- equation → (A.1), (A.2), ...
- problem → A.1, A.2, ...
- theorem → A.1, A.2, ...
- algorithm → A.1, A.2, ...

A.2 Section Heading Format

Section headings inside the appendix are automatically formatted as **Appendix A**, **Appendix B**, and so on rather than just **A** or **1**. Subsections and subsubsections are not affected and continue to number normally (A.1, A.1.1, and so on).

A.3 Cross-Referencing Appendices

Appendix sections, subsections, figures, and tables all support `\label` and `\cref` the same way as the rest of the document. For example, this subsection has the label `sec:appendix-usage-crossref` and can be referenced anywhere in the document like this:

Listing A.2: Appendix Section Cross-Reference.

```
1 % In the appendix:  
2 \subsection{Cross-Referencing Appendices}\label{sec:appendix-crossref}  
3  
4 % Elsewhere in the document:  
5 See \cref{sec:appendix-crossref} for details on cross-referencing  
   ↪ appendices.
```

Which renders as: See appendix [A.3](#) for details on cross-referencing appendices.

Appendix B Metadata Reference

All document settings are controlled through the metadata block at the top of `main.tex`. Each field is defined with `\newcommand` and read by the template when the document compiles. This appendix lists every available field, what values it accepts, and what it changes in the document.

B.1 Submission Information

These fields populate the title page and page header. Update them for every new submission.

Table B.1: Submission metadata fields.

Field	Description
<code>\metaAssignmentTitle</code>	The assignment or document title shown on the title page.
<code>\metaStudentName</code>	Your full name. Appears on the title page and in the page header when <code>\metaPartners</code> is empty.
<code>\metaStudentNumber</code>	Your student number. Shown in brackets after your name in the header when <code>\metaPartners</code> is empty.
<code>\metaStudentEmail</code>	Your university email address. Shown on the title page.
<code>\metaPartners</code>	Group partners listed as <code>Name & Number\Name & Number</code> . When non-empty, the student name and number are removed from the header. Leave empty for solo submissions.
<code>\metaUniversity</code>	University name shown on the title page.
<code>\metaFaculty</code>	Faculty name shown on the title page.
<code>\metaDepartment</code>	Department name shown on the title page. Leave empty to omit.
<code>\metaCourseCode</code>	Course code (e.g. EECS 3000). Always shown in the left header.
<code>\metaCourseName</code>	Full course name. Used by the smart header logic to decide whether to show the full name, a shortened acronym, or nothing depending on available space.
<code>\metaInstructor</code>	Instructor name shown on the title page.

Field	Description
<code>\metaDate</code>	Submission or due date shown on the title page.

B.2 Title Page

Table B.2: Title page metadata fields.

Field	Description
<code>\metaTitlePage</code>	Controls the title page layout. Accepted values: <code>standard</code> (full title page with all fields), <code>abstract</code> (includes the <code>\metaAbstract</code> field below the title block), <code>compact</code> (condensed single-page layout).
<code>\metaLogoPath</code>	Path to the faculty or university logo image. Leave empty to omit the logo from the title page.
<code>\metaLogoLocation</code>	Controls the location of the logo from the <code>\metaLogoPath</code> in the standard title page. The location can either be centered (default), or in the bottom right-hand corner (<code>york</code> , York University configuration).
	<i>Located below Drafting metadata macros</i>
<code>\metaAbstract</code>	Abstract text used when <code>\metaTitlePage</code> is set to <code>abstract</code> . Has no effect for other title page styles.

B.3 Bibliography

Table B.3: Bibliography metadata fields.

Field	Description
<code>\metaBibFile</code>	Filename of the <code>.bib</code> reference file (e.g. <code>references.bib</code>).
<code>\metaBibStyle</code>	Citation and bibliography style. Example values: <code>ieee</code> , <code>apa</code> . See section 19.2 for a full list of inputs.

Field	Description
<code>\metaShowBackref</code>	When true, each bibliography entry shows the page numbers where it was cited. Back-references are coloured using <code>\metaUrlColour</code> . Accepted values: true, false.

B.4 Font & Spacing

Table B.4: Font and spacing metadata fields.

Field	Description
<code>\metaSerifFont</code>	Serif font loaded by the template. Leave empty for IBM Plex Serif, or set to any system font name (e.g. Georgia). Used by <code>\seriftext{}</code> .
<code>\metaSansFont</code>	Sans-serif font loaded by the template. Leave empty for IBM Plex Sans, or set to any system font name (e.g. Arial). Used by <code>\sansseriftext{}</code> .
<code>\metaDefaultFont</code>	Document body font family. Accepted values: sans (uses <code>\metaSansFont</code>), serif (uses <code>\metaSerifFont</code>).
<code>\metaLineSpacing</code>	Line spacing throughout the document. Accepted values: single, onehalf, double.

B.5 Colours

Each field accepts a hex colour code (with # before it), a built-in $\text{\LaTeX}$ colour name, or a custom colour defined in the template. Leave any field empty to fall back to the default. See tables 2.2 and 2.3 for available colour names.

Table B.5: Colour metadata fields.

Field	Description
<code>\metaThemeColour</code>	Primary accent colour. Controls headings, rules, boxes, line numbers, and figure labels. Defaults to <code>DefaultBlue</code> when empty.
<code>\metaLinkColour</code>	Colour for internal cross-references and hyperlinks. Leave empty to use <code>themecolour</code> .

Field	Description
<code>\metaCiteColour</code>	Colour for citation references. Leave empty to use <code>themecolour</code> .
<code>\metaUrlColour</code>	Colour for URLs and bibliography back-references. Leave empty to use <code>themecolour</code> .

B.6 Numbering

These fields control whether floats and environments are numbered by section (e.g. Fig. 2.1) or continuously through the whole document (e.g. Fig. 3). All fields accept `true` or `false`.

Table B.6: Numbering metadata fields.

Field	Affected Environment
<code>\metaNumberEquations</code>	equation, reaction
<code>\metaNumberFigures</code>	figure
<code>\metaNumberTables</code>	table
<code>\metaNumberTheorems</code>	All theorem environments
<code>\metaNumberProblems</code>	problem
<code>\metaNumberAlgorithms</code>	algorithm
<code>\metaNumberListings</code>	listing

B.7 Drafting

These fields controls aspects of the document to increase compile time during drafting. It is recommended to have these set to `false` while drafting and `true` only for a final copy.

Table B.7: Minted metadata field.

Field	Description
<code>\metaFinalDraft</code>	<code>true</code> will turn on packages and settings which slow compilation but improve overall document quality. <code>false</code> decreases compile times by turning these settings off.

Field	Description
<code>\metaShowFigures</code>	<code>true</code> will turn on the ability to see images inserted with <code>\includegraphics</code> and vector images using <code>tikz</code> (including <code>problem/solution</code> environments). <code>false</code> produces a placeholder, allowing faster compile times.
<code>\metaShowBib</code>	<code>true</code> will turn on the bibliography and citations. <code>false</code> reduces compile time by outputting a red citation key for citations, and a placeholder for the bibliography.
<code>\metaUseMinted</code>	Allows use of <code>minted</code> package for listing environments. Input <code>true</code> to use, <code>false</code> to remove access. See section 17.2 for full instructions.

Appendix C Side-by-Side Layouts with `minipage`

The `minipage` environment is a built-in $\text{\LaTeX}$ tool that creates a self-contained box of a fixed width, like a small independent column on the page. No package is required. It is the most reliable way to place two pieces of content next to each other, especially when one of them contains a display math equation, since equations are not supported inside table cells.

Place two `minipages` side by side by putting `\hfill` between them. `\hfill` pushes the two boxes to opposite sides of the line. Make sure the two widths add up to less than `\linewidth` so there is room for the fill in between.

The optional alignment argument controls how the two `minipages` line up vertically: `[t]` aligns their tops, `[b]` aligns their bottoms, and `[c]` (the default) aligns their centres.

C.1 Text Side by Side

Left panel. This `minipage` occupies 45% of the line width. Content wraps independently within its column and does not affect the right panel.

Right panel. `\hfill` between the two `minipages` pushes them to opposite sides. Adjust the widths so they sum to less than `\linewidth` to leave room for the fill.

C.2 Display Math Side by Side

A common use case is showing two forms of the same expression next to each other for easy comparison:

Expanded form:

$$f(x) = x^2 + 2x + 1$$

Factored form:

$$f(x) = (x + 1)^2$$

C.3 Proof Step Comparison

`Minipages` are also useful for showing a simplification step before and after on the same line:

Before:

$$\frac{x^2 - 1}{x - 1} = \frac{(x - 1)(x + 1)}{x - 1}$$

After (for $x \neq 1$):

$$= x + 1$$

C.4 Figure and Caption Side by Side

`Minipages` can also place two figures side by side without `subcaption`, when you want each figure to have its own independent caption and label rather than sharing a parent

figure:



Figure C.1: First independent figure.



Figure C.2: Second independent figure.

Note: `\captionof` from the `caption` package is required to place a caption outside a figure float environment.

Appendix D Notation

This appendix lists the mathematical notation used throughout this document, grouped by subject area. It is an example of how you might document your own notation in a report or assignment.

D.1 General Mathematics

Table D.1: General mathematical notation.

Symbol	Meaning
$f(x), g(x)$	Functions of x
$f(x) := \dots$	Definition of $f(x)$ (defined as equal to)
$\mathbb{R}, \mathbb{Z}, \mathbb{N}$	Real numbers, integers, natural numbers
∞	Infinity
$\sum_{n=1}^N$	Sum of terms from $n = 1$ to N
$\lfloor x \rfloor$	Floor of x , the largest integer less than or equal to x
$ x $	Absolute value of x

D.2 Algebra

Table D.2: Algebra notation.

Symbol	Meaning
$a^2 + b^2 = c^2$	Pythagorean theorem
$(a + b)^2 = a^2 + 2ab + b^2$	Binomial expansion
$ x = \begin{cases} x & x \geq 0 \\ -x & x < 0 \end{cases}$	Absolute value as a piecewise function
$\mathbf{v}, \mathbf{u}$	Vectors (bold italic)
A, B	Matrices (uppercase letters)

D.3 Physics

Table D.3: Physics notation.

Symbol	Meaning
$\mathbf{F}$	Force vector (N)
m	Mass (kg)
$\mathbf{a}$	Acceleration vector (m/s ²)
$\mathbf{v}$	Velocity vector (m/s)
t	Time (s)
V	Voltage (V)
I	Current (A)
R	Resistance (Ω)

Appendix E Greek Letters Reference

A complete reference of Greek letters available in math mode. Greek letters are used often in science and engineering for variables, constants, and operators.

Table E.1: Lowercase Greek letters.

Symbol	Command	Symbol	Command	Symbol	Command
α	<code>\alpha</code>	ι	<code>\iota</code>	ρ	<code>\rho</code>
β	<code>\beta</code>	κ	<code>\kappa</code>	σ	<code>\sigma</code>
γ	<code>\gamma</code>	λ	<code>\lambda</code>	τ	<code>\tau</code>
δ	<code>\delta</code>	μ	<code>\mu</code>	υ	<code>\upsilon</code>
ϵ	<code>\epsilon</code>	ν	<code>\nu</code>	ϕ	<code>\phi</code>
ε	<code>\varepsilon</code>	ξ	<code>\xi</code>	φ	<code>\varphi</code>
ζ	<code>\zeta</code>	π	<code>\pi</code>	χ	<code>\chi</code>
η	<code>\eta</code>	ϖ	<code>\varpi</code>	ψ	<code>\psi</code>
θ	<code>\theta</code>	ς	<code>\varsigma</code>	ω	<code>\omega</code>
ϑ	<code>\vartheta</code>				

Table E.2: Uppercase Greek letters (only those that differ from Latin capital letters).

Symbol	Command	Symbol	Command	Symbol	Command
Γ	<code>\Gamma</code>	Λ	<code>\Lambda</code>	Φ	<code>\Phi</code>
Δ	<code>\Delta</code>	Ξ	<code>\Xi</code>	Ψ	<code>\Psi</code>
Θ	<code>\Theta</code>	Π	<code>\Pi</code>	Ω	<code>\Omega</code>
Σ	<code>\Sigma</code>	Υ	<code>\Upsilon</code>		

Appendix F Common Math Operators and Symbols

F.1 Logic and Sets

Table F.1: Logic and set notation.

Symbol	Command	Symbol	Command
$\Rightarrow$	<code>\implies</code>	$\in$	<code>\in</code>
$\Leftrightarrow$	<code>\iff</code>	$\notin$	<code>\notin</code>
$\forall$	<code>\forall</code>	$\subset$	<code>\subset</code>
$\exists$	<code>\exists</code>	$\subseteq$	<code>\subseteq</code>
$\neg$	<code>\neg</code>	$\cup$	<code>\cup</code>
$\wedge$	<code>\land</code>	$\cap$	<code>\cap</code>
$\vee$	<code>\lor</code>	$\emptyset$	<code>\emptyset</code>
$\oplus$	<code>\oplus</code>	$\setminus$	<code>\setminus</code>

F.2 Relations

Table F.2: Relation symbols.

Symbol	Command	Symbol	Command
$\equiv$	<code>\equiv</code>	$\approx$	<code>\approx</code>
$\cong$	<code>\cong</code>	$\sim$	<code>\sim</code>
$\propto$	<code>\propto</code>	$\leq$	<code>\leq</code>
$\neq$	<code>\neq</code>	$\geq$	<code>\geq</code>
$\ll$	<code>\ll</code>	$\gg$	<code>\gg</code>

F.3 Arrows

Table F.3: Arrow symbols.

Symbol	Command	Symbol	Command
$\rightarrow$	<code>\rightarrow</code>	$\leftarrow$	<code>\leftarrow</code>
$\Rightarrow$	<code>\Rightarrow</code>	$\Leftarrow$	<code>\Leftarrow</code>
$\Leftrightarrow$	<code>\Leftrightarrow</code>	$\mapsto$	<code>\mapsto</code>
$\uparrow$	<code>\uparrow</code>	$\downarrow$	<code>\downarrow</code>

Appendix G Font Commands

G.1 Text Font Shapes and Weights

Table G.1: Text font commands.

Command	Output	Common use
<code>\textbf{...}</code>	Bold text	Emphasis, key terms
<code>\textit{...}</code>	<i>Italic text</i>	Titles, emphasis
<code>\texttt{...}</code>	Monospace text	Code, file names, commands
<code>\textsc{...}</code>	Small Caps	Acronyms, proper names
<code>\textsf{...}</code>	Sans-serif text	Labels, captions
<code>\emph{...}</code>	<i>Emphasised text</i>	Context-aware italic
<code>\underline{...}</code>	<u>Underlined text</u>	Use sparingly

G.2 Font Sizes

Font size commands are applied by placing them inside a group with curly braces: `{\large This text is large.}`. They scale relative to the base font size of the document.

Table G.2: Font size commands.

Command	Output
<code>\tiny</code>	tiny text
<code>\scriptsize</code>	script size
<code>\footnotesize</code>	footnote size
<code>\small</code>	small text
<code>\normalsize</code>	normal text
<code>\large</code>	large text
<code>\Large</code>	Large text
<code>\LARGE</code>	LARGE text
<code>\huge</code>	huge text